

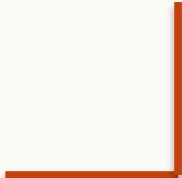
Attention Is All You Need



A guided tour of the Transformer, sequence transduction with attention only

Vaswani · Shazeer · Parmar · Uszkoreit · Jones · Gomez · Kaiser · Polosukhin

WMT14 · EN→DE 28.4 BLEU · EN→FR 41.8 BLEU · 8× P100 · 3.5 days



Sequential computation caps RNNs

Serial by position

$h(t)$ depends on $h(t-1)$: no parallelism within a training example, long sequences stall.

Distance hurts

In ConvS2S / ByteNet the ops linking two positions grow linearly or log with distance.

Attention as sidekick

Attention already worked, but almost always on top of an RNN. The bottleneck stayed.

The move The Transformer drops recurrence and convolution entirely: any two positions connect in a constant number of steps.

Attention only: faster, and better

The first sequence transduction model built entirely on attention. Far more parallel training, and higher translation quality.

28.4 BLEU

WMT14 EN→DE, beats all prior models and ensembles by 2+ BLEU

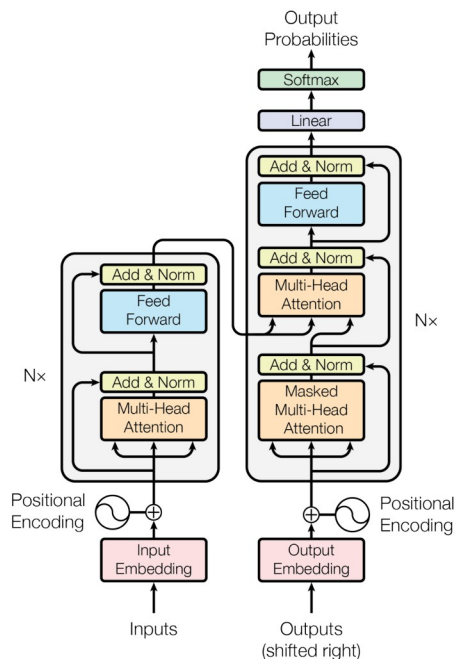
41.8 BLEU

WMT14 EN→FR, new single-model state of the art

3.5 days

on 8× P100 for the big model; base tops all published EN→DE models in 12 hours

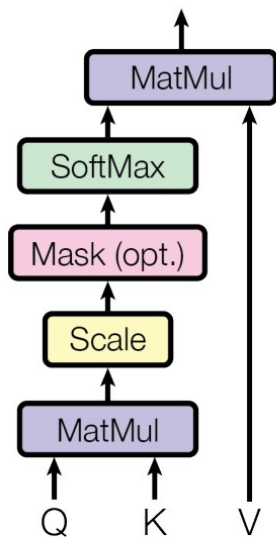
Six encoder + six decoder layers



Paper Figure 1 (original)

- **Encoder x6:** each layer = multi-head self-attention + position-wise feed-forward
- **Decoder x6:** adds encoder-decoder attention; self-attention is masked to the generated prefix
- **Residual + LayerNorm:** every sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$; $d_{\text{model}} = 512$ throughout
- **Output:** linear layer + softmax over the vocabulary

One matmul computes all attention weights

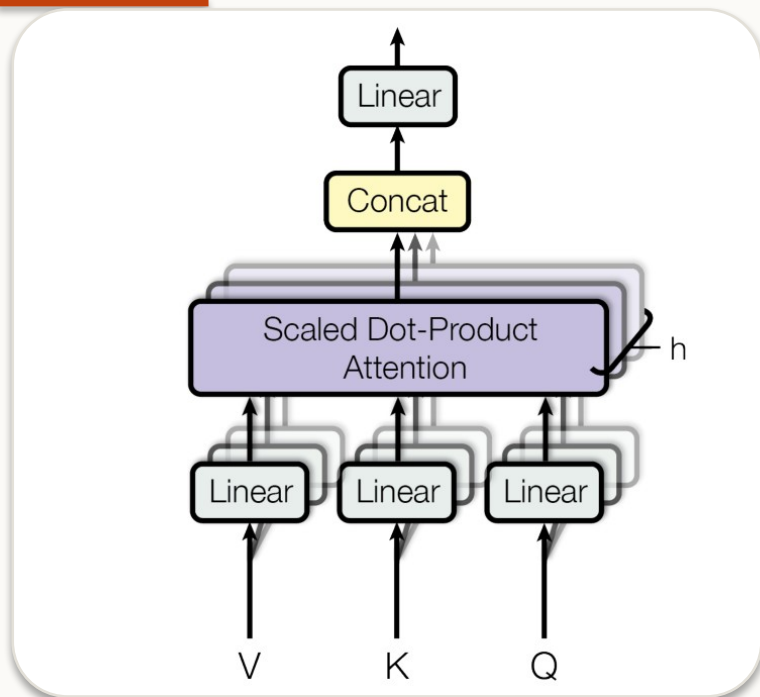


Paper Figure 2, left

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- **Q·K sets weights:** dot products of a query with all keys, softmaxed into weights over values V
- **Divide by $\sqrt{d_k}$:** large dims push softmax into tiny-gradient regions; scaling undoes it
- **Fast and lean:** the whole sequence is packed into matrices, one highly-optimized matmul

Multi-head: 8 parallel subspaces



Paper Figure 2, right

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- **$h = 8$ heads:** project Q, K, V eight times, attend in parallel, concat and project
- **$dk = dv = 64$ per head:** dims shrink per head, total cost $\approx$ single-head full dimension
- **Why heads at all:** one head averages information away; heads watch different subspaces

One mechanism, used in three places



Encoder self-attention

Q, K, V all come from the previous encoder layer. Every position sees the whole sentence.



Masked decoder self-attention

Positions see only the prefix; illegal links get $-\infty$ before softmax, keeping it auto-regressive.



Encoder-decoder attention

Q from the decoder, K and V from the encoder. Every decoding step queries the full input.

The rest: FFN and positional encoding

Position-wise feed-forward

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

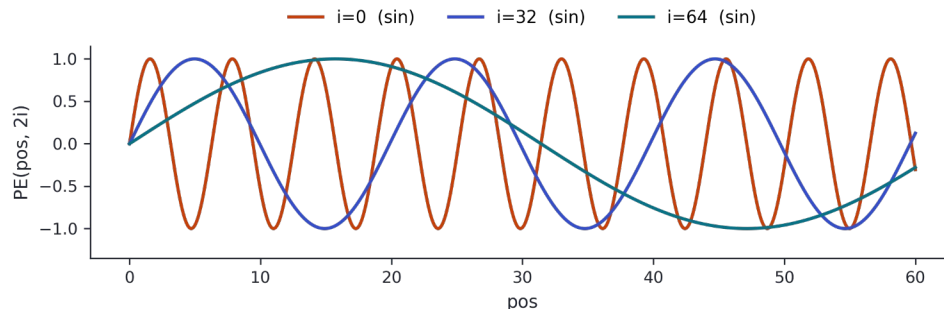
Each position passes through the same two-layer net independently; inner size $d_{\text{ff}} = 2048$.

Sinusoidal positional encoding

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Order comes from sinusoids added to the embeddings, one wavelength per dimension (right). Fixed offsets are linear maps; learned embeddings do equally well.



Why self-attention: the shortest paths

Layer type	Complexity / layer	Sequential ops	Max path length
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

Takeaway Any two positions connect in one step ($O(1)$ path) with constant sequential ops; and since sentence representations usually have $n < d$, self-attention is cheaper per layer too.

Training recipe: 8× P100, at most 3.5 days

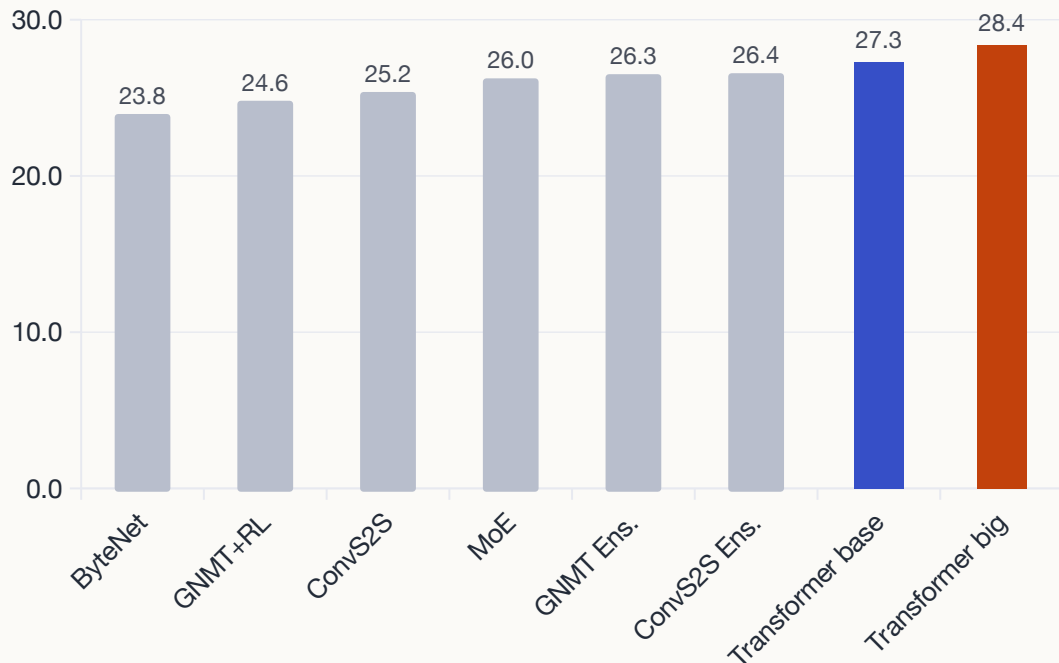
Data & tokenization

EN→DE	WMT14, 4.5M sentence pairs
EN→FR	WMT14, 36M sentences
Tokens	BPE shared vocab 37K / word-piece 32K
Batch	≈25K source + 25K target tokens per batch

Hardware & optimization

Hardware	one machine · 8× NVIDIA P100
Schedule	base 100K steps ≈ 12 h · big 300K steps ≈ 3.5 days
Optimizer	Adam, warmup 4000 steps then inverse-sqrt decay
Regularization	residual dropout 0.1 · label smoothing 0.1

EN→DE: base beats all, big sets the record



WMT14 EN→DE test BLEU (paper Table 2)

AT WHAT COST?**1/7**

Base needs ~1/7 of GNMT+RL's training compute (3.3 vs 23×10^{18} FLOPs) yet gains 2.7 BLEU; on EN→FR the big model hits 41.8 at a fraction of prior SOTA cost.

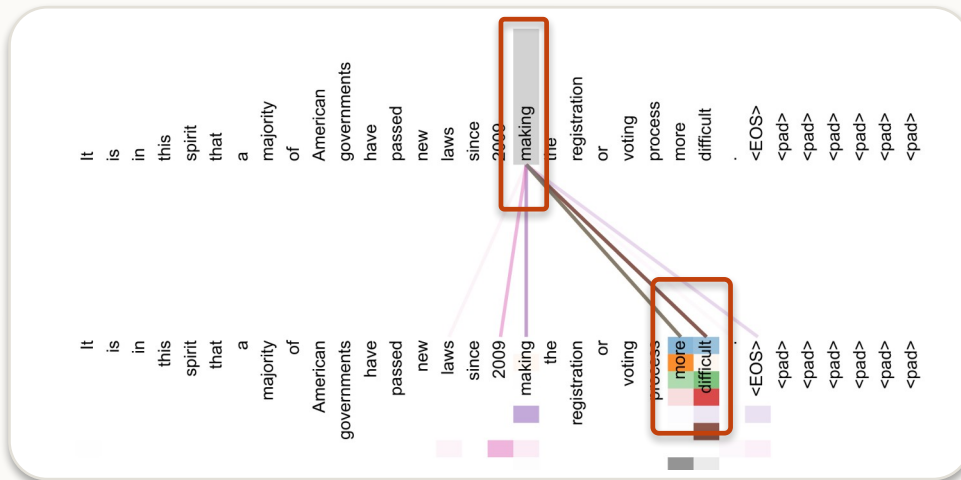
Ablations: heads and dropout matter most

Configuration	dev BLEU
base (8 heads, dk=64)	25.8
single head	24.9
32 heads	25.4
no dropout	24.6
learned pos. embedding	25.7
big (wider & deeper)	26.4

- **Heads have a sweet spot:** one head is 0.9 BLEU worse than best; 32 heads also degrade
- **Don't shrink dk:** smaller key size hurts; compatibility is not trivial
- **Bigger + dropout:** bigger models win; removing dropout costs 1.2 BLEU
- **PE choice is free:** learned $\approx$ sinusoidal (25.7 vs 25.8)

It parses too, and attention is readable

F1 92.7 English constituency parsing (WSJ): F1 91.3 with only 40K training sentences, beating BerkeleyParser; 92.7 semi-supervised, above all prior work except RNNG.



An encoder head in layer 5: “making” attends across the phrase to “more difficult” (paper Figure 3, original).

The first attention-only transduction model

- **Recurrence replaced:** multi-head self-attention fully replaces the recurrent layers of encoder-decoder models
- **Faster + SOTA:** trains much faster than recurrent/conv architectures; new records on EN→DE and EN→FR
- **Next (their words):** extend to images, audio, video; local restricted attention; less sequential generation

Code released at [tensorflow/tensor2tensor](https://github.com/tensorflow/tensor2tensor).

It went on to change everything



Presenter's extension. Not part of the paper.

Thanks · Q&A